

What we're going to cover:

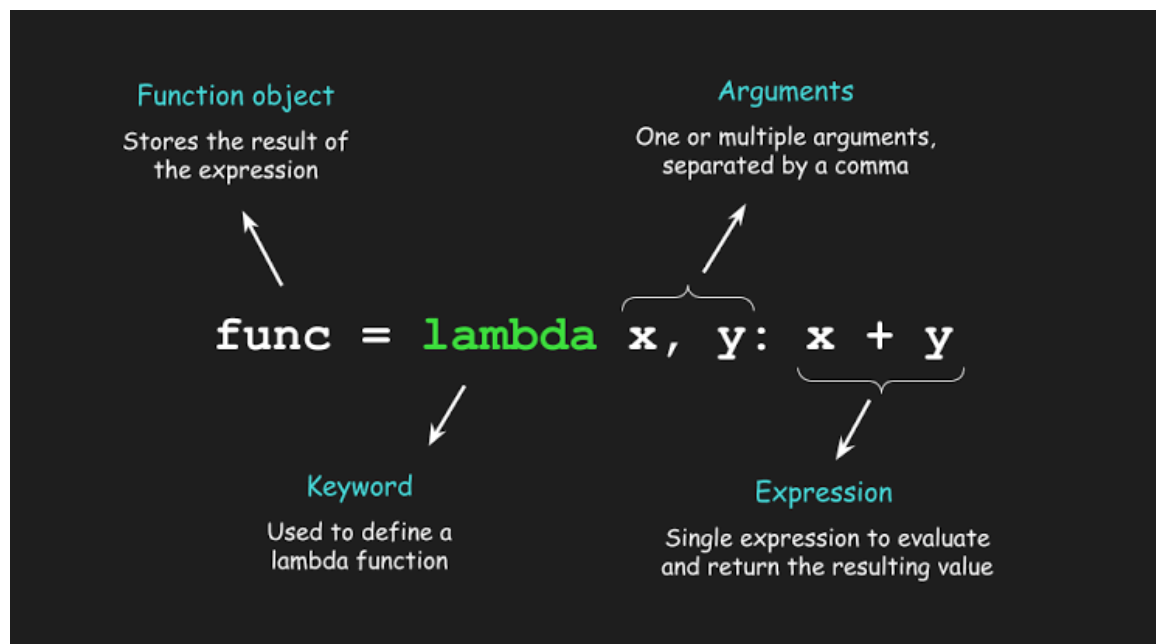
- Lambda function (anonymous function)
- Map
- Zip
- Filter
- Reversed
- Homework for student (difference between sort and sorted)

Lambda function

تابع لامبدا چیست؟

تابع لامبدا یک تابع کوچک و تک خطی است که بدون نام تعریف می شود

این توابع می توانند هر تعداد پارامتر ورودی داشته باشند، اما تنها شامل یک عبارت هستند؛ نتیجه ی همین عبارت نیز به عنوان خروجی تابع بازگردانده می شود



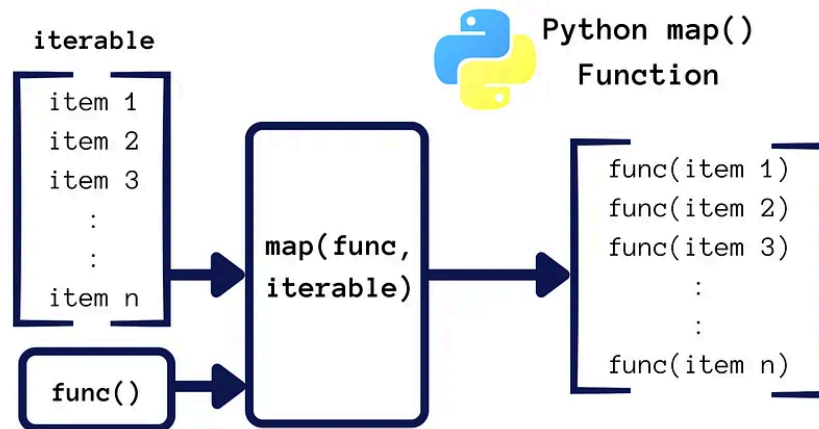
Reference: Medium

```
In [1]: def short_function(x):  
        return x * 2  
  
        # Rather than using function we can use Lambda  
        equiv_anon = lambda x: x * 2
```

```
In [10]: x = lambda a, b : a * b  
         print(x(5, 6))
```

```
In [8]: check = lambda x: "Positive" if x > 0 else "Negative" if x < 0 else "Zero"
print(check(5))
print(check(-3))
print(check(0))
```

Positive
Negative
Zero



map()

این تابع، یک تابع مشخص را برای تک تک عناصر یک شیء قابل تکرار (مانند لیست‌ها، تاپل‌ها و غیره) اجرا می‌کند

Syntax: map(function, iterables)

این تابع دو آرگومان می‌گیرد:

- function

تابعی که بر تک تک عناصر یک شیء قابل تکرار اعمال می‌شود

- iterables

اشیاء قابل تکرار مانند لیست‌ها، تاپل‌ها و غیره

مزایای استفاده از Map نسبت به Loop

- map()
- هنگام کار با مجموعه‌های بزرگ، معمولاً سریع‌تر عمل می‌کند
- loops
- معمولاً کندتر هستند، زیرا مستلزم آن‌اند که مفسر دستور حلقه را برای هر تکرار اجرا کند

```
In [4]: numbers = [1,2,3,4]

# returns the square of a number
```

```
def square(number):
    return number * number

# apply square() to each item of the numbers list
squared_numbers = map(square, numbers)

# converting to list for printing
result = list(squared_numbers)
print(result)
```

[1, 4, 9, 16]

map() with Lambda

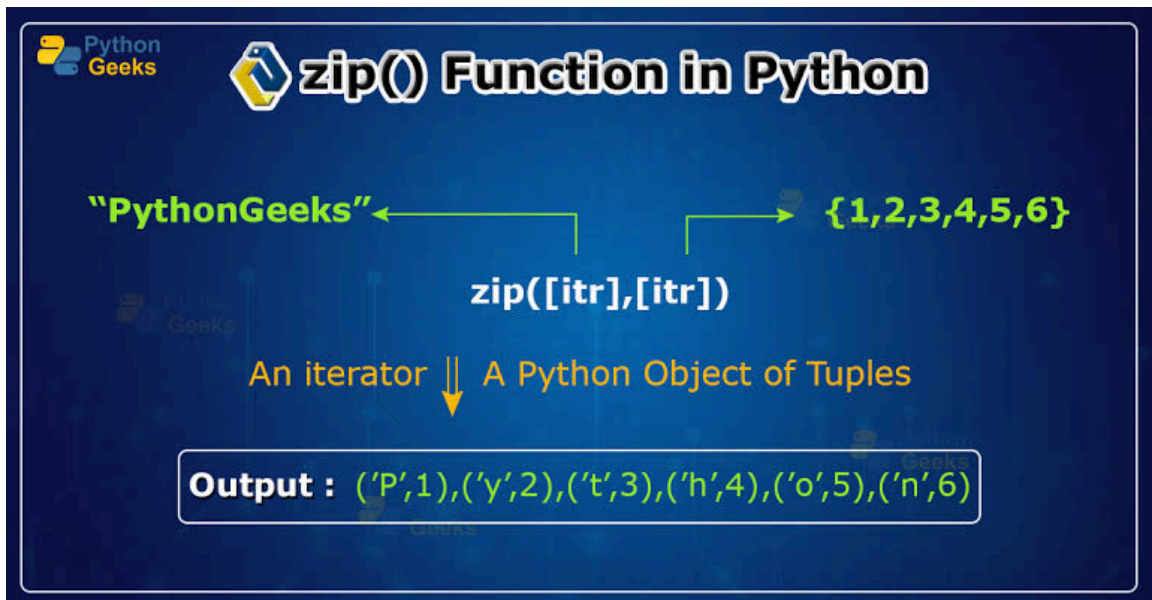
```
In [8]: numbers = (1, 2, 3, 4)
result = map(lambda x: x*x, numbers)
print(result)

# convert to list and print it
print(list(result))
```

<map object at 0x000001ADE05A43A0>
[1, 4, 9, 16]

```
In [1]: ages = [25, 32, 45, 60, 75]
age_bins = list(map(lambda x: 'young' if x < 35 else ('middle-aged' if x < 60 else
print(age_bins)
```

['young', 'young', 'middle-aged', 'senior', 'senior']

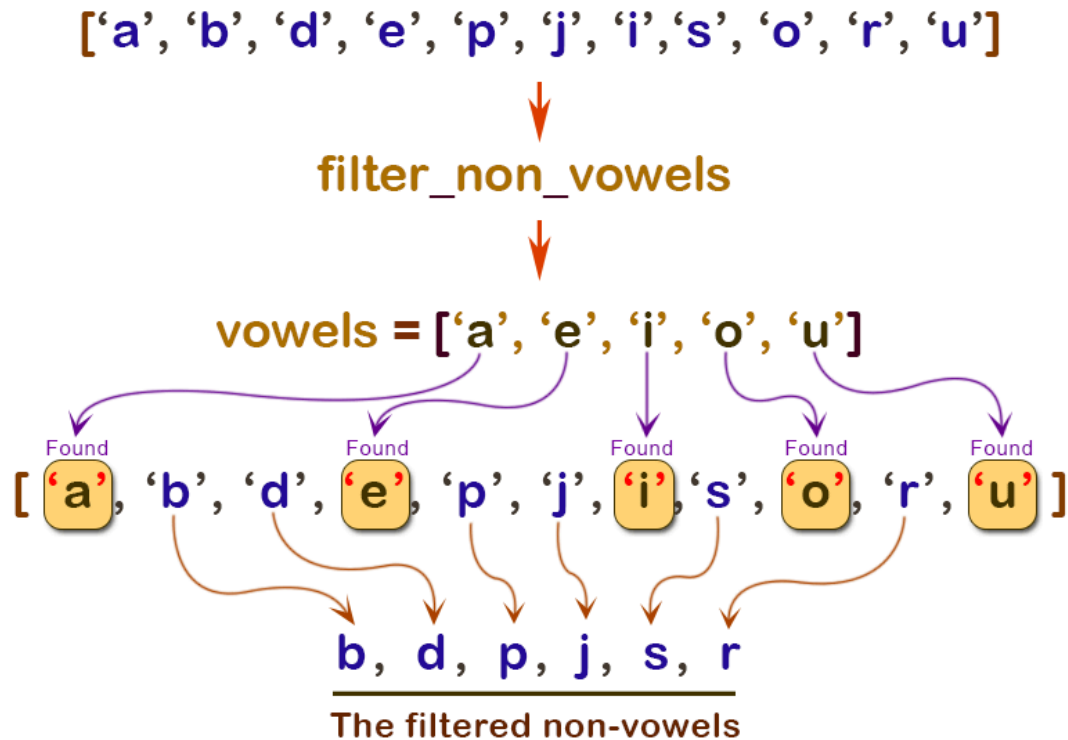


zip

عناصر تعدادی از لیست‌ها، تاپل‌ها یا سایر دنباله‌ها را با هم جفت می‌کند تا لیستی از تاپل‌ها ایجاد کند

```
In [2]: seq1 = ["foo", "bar", "baz"]
seq2 = ["one", "two", "three"]
zipped = zip(seq1, seq2)
list(zipped)
```

```
Out[2]: [('foo', 'one'), ('bar', 'two'), ('baz', 'three')]
```



© w3resource.com

filter()

در پایتون، از این تابع برای جدا کردن (فیلتر کردن) عناصر یک شیء قابل تکرار (مانند لیست، تاپل و غیره) بر اساس یک شرط استفاده می شود

این تابع، یک تابع دیگر را روی تک تک عناصر آن مجموعه اعمال می کند و تنها عناصری در نتیجه نهایی قرار می گیرند که تابع مذکور برای آن ها مقدار درست را برگرداند.

Syntax: filter(function, iterables)

این تابع دو آرگومان می گیرد :

- **function**
- تابعی که برای هر عنصر در شیء قابل تکرار مقدار درست یا غلط را برمی گرداند
- **iterable**
- آن شیء قابل تکرار (مانند لیست یا تاپل) است که می تواند عناصرش را فیلتر کند

```
In [5]: numbers = [1, 2, 3, 4, 5, 6, 7]
```

```
def is_even(x):
```

```
return x % 2 == 0
```

```
even_numbers = filter(is_even, numbers)  
print(list(even_numbers))
```

[2, 4, 6]

filter() with a Lambda

In [6]: `numbers = [1, 2, 3, 4, 5, 6, 7]`

```
even_numbers = list(filter(lambda x: x % 2 == 0 , numbers))  
print(even_numbers)
```

[2, 4, 6]

reversed

روی عناصر یک دنباله به ترتیب معکوس پیمایش می‌کند :

In [3]: `list(reversed(range(10)))`

Out[3]: [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

Homework:

What is the difference between `sort()` and `sorted()` ?